

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

UNIT III

COLLABORATIVE FILTERING

UNIT III

COLLABORATIVE FILTERING

A systematic approach, Nearest-neighbour collaborative filtering (CF), user-based and item-based CF, components of neighbourhood methods (rating normalization, similarity weight computation, and neighbourhood selection.

A systematic approach - Nearest-neighbour collaborative filtering (CF)

Collaborative Filtering (CF) methods collect preferences in the form of ratings or signals from many users (hence the name) and then recommend items to a user based on item interactions that people have with similar tastes as this user had in the past. In other words, these methods assume that if person X likes a subset of items that person Y likes, then X is more likely to have the same opinion as Y for a given item compared to a random person that may or may not have the same preferences.

The main idea with neighborhood-based methods is to leverage either user-user similarity or item-item similarity to make recommendations. These methods assume that similar users tend to have similar behaviors when rating items. We can also expand this assumption to items as well: similar items tend to receive similar ratings from the same user.

In these methods, the interactions between users and items are generally represented by a user-item matrix, where each row represents a user and each column represents an item, while the cells represent the interaction between the two, which, in most cases, are the item ratings made by users. In this context, we can define two types of neighborhood-based methods:

- **User-based Collaborative Filtering:** Ratings given by users like a user U are used to make recommendations. More specifically, to predict U's rating for a given item I, we calculate the weighted average of the rating r of k similar users (neighbors) to U, where the weights are determined by the similarity between U and each of the similar users.

$$\hat{r}_{ui} = \frac{\sum_{v \in N_i^k(u)} \text{sim}(u, v) \cdot r_{vi}}{\sum_{v \in N_i^k(u)} \text{sim}(u, v)}$$

- **Item-based Collaborative Filtering:** Ratings of a group of similar items are used to make recommendations for item I. Similarly, to predict I's rating given by a user U, we calculate the weighted average of the rating r of k similar items (neighbors) to I, where the weights are determined by the similarity between I and each of the similar items.

$$\hat{r}_{ui} = \frac{\sum_{j \in N_u^k(i)} \text{sim}(i, j) \cdot r_{uj}}{\sum_{j \in N_u^k(i)} \text{sim}(i, j)}$$

Comparison between User-based and Item-based Methods

The difference is subtle, but user-based collaborative filtering predicts a user's rating by using the ratings of neighboring users, while item-based collaborative filtering leverages the user's ratings on neighboring items, which allows for more consistent predictions because it follows the rating behaviors of that user. In the former case, the similarity is calculated between the rows of the user- item matrix, while the latter looks at similarities between the columns of the matrix.

These approaches also differ in the way they solve problems. It is common to use the item neighborhood to recommend a list of top k items to a user. On the other hand, it is interesting to retrieve the top k users from a segment to target them for marketing campaigns.

To understand the reasoning behind a recommendation, item-based methods provide better explanations than user-based methods. This is because item-based recommendations can use the item neighborhood to explain the results in the form of "you bought this, so these are the recommended items". The item neighborhood can also be useful for suggesting product bundles to maximize sales. On the other hand, user-based methods' recommendations usually cannot be explained directly because neighbor users are anonymized for privacy reasons.

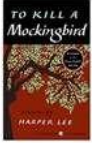
Additionally, item-based methods may only recommend items very similar to what the user already liked, whereas user-based methods often recommend a more diverse set of items. This can encourage users to try new items and potentially keep their engagement and interest.

Another significant difference between these approaches is related to ratings. Calculating the similarity between users to predict ratings may be misleading because users may rate items in a different manner. When you present a range of values to the user, he/she might interpret them differently. For instance, in a 5-star rating system, a user may rate an item as 3 because it does what it is expected to do and nothing more, while others might use 3 to rate an item that barely works. Some users rate items highly and others rate items less favorably. To address this issue, the ratings should be mean centered by the user, meaning the user's mean rating is subtracted from their raw rating, and the target user's mean rating is added to the calculation, as in the example below:

$$\hat{r}_{ui} = \mu_u + \frac{\sum_{v \in N_i^k(u)} \text{sim}(u, v) \cdot (r_{vi} - \mu_v)}{\sum_{v \in N_i^k(u)} \text{sim}(u, v)}$$

Neighborhood Models in Practice

Let's say we have the following small sample of a user-item matrix, where items are from a digital commerce store. Notice there are missing ratings, which means users typically do not rate all products.

						
Emily	4	4	4	5	-	-
Michael	3	4	3	-	2	5
Madison	3	1	-	3	4	-
Matthew	5	2	-	-	4	-
Olivia	5	3	3	4	3	2

Note: Product images extracted from Amazon Marketplace

To show how the algorithm works in practice, let's assume we have built an item-based model. Note that the steps of the algorithm would be analogous to the user-based model, except for the perspective changes and focus on similarities between rows (users).

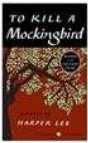
Remember that neighborhood CF algorithms rely on the ratings and similarity between items/users, so the first step is to define which similarity metric to use. One of the most common choices is the Pearson similarity, which measures how correlated a pair of vectors are. The range of values scales from -1 to 1, where those values indicate negative and positive correlations, respectively, and 0 indicates no correlation between vectors. This is the Pearson similarity equation for item-based models:

$$\text{pearson_sim}(i, j) = \frac{\sum_{u \in U_{ij}} (r_{ui} - \mu_i) \cdot (r_{uj} - \mu_j)}{\sqrt{\sum_{u \in U_{ij}} (r_{ui} - \mu_i)^2} \cdot \sqrt{\sum_{u \in U_{ij}} (r_{uj} - \mu_j)^2}}$$

During this first phase, it's usual to precompute the similarity matrix beforehand to obtain a good performance during inference time. In the case of item-based models, an item-item similarity matrix is built by applying the similarity metric between all pairs of items. Since the matrix is sparse, we only consider the set of mutually rated pairs of items during the similarity computation. For instance, the similarity between items from columns 1 and 4 of the image above will be computed as the similarity between vectors [4,3,5] and [5,3,4]. It's possible that a pair of items may show no co-ratings by users due to the sparsity of the matrix, resulting in an empty set. In that case, a value of 0 similarity is assigned for that pair. To improve computational efficiency, it is common to consider only the k nearest neighbors of an item during inference time.

Let's say we want to predict how Madison rated the Animal Farm book and we defined $k=2$ as the number of nearest neighbors to consider during calculation. To simplify the example, we will only manually calculate the similarities between the target item and items from columns 2 and 4 because they are the nearest neighbors for this item. When calculating the mean rating during similarity computation, we will consider only the set of ratings that are mutually exclusive between items.

The image below shows how the neighborhood is formed. The circle in red is the value we're trying to predict. The squares in green are ratings from Madison that are going to be used to infer the rating for the target item. The other two ratings marked with an X are not considered because $k=2$. The rectangles in orange show a set of mutually exclusive ratings between the target item and the item from column 2, while the rectangles in blue show the same, but for the common ratings between the target item and item from column 4.

						
Emily	4	4	4	5	-	-
Michael	3	4	3	-	2	5
Madison		1	-	3		-
Matthew	5	2	-	-	4	-
Olivia	5	3	3	4	3	2

These are the common set of ratings between the target item (item 3) and the first neighbor (item 2): [4,3,3] and [4,4,3]. The first step is to calculate the mean in each set:

$$\text{Target item mean} = \frac{4 + 3 + 3}{3} = 3.33$$

$$\text{Item 2 mean} = \frac{4 + 4 + 3}{3} = 3.67$$

The Pearson similarity formula centers the ratings by their mean, so we can transform this vector and then plug the results into the equation:

$$\text{Target item mean centered vector} = [(4 - 3.33), (3 - 3.33), (3 - 3.33)] = [0.67, -0.33, -0.33]$$

$$\text{Item 2 mean centered vector} = [(4 - 3.67), (4 - 3.67), (3 - 3.67)] = [0.33, 0.33, -0.67]$$

To simplify the calculations, we separate the numerator and denominator:

$$\text{Numerator} = (0.67 * 0.33) + (-0.33 * 0.33) + (-0.33 * -0.67) = 0.33$$

$$\text{Denominator} = \sqrt{0.67^2 + (-0.33)^2 + (-0.33)^2} * \sqrt{0.33^2 + 0.33^2 + (-0.67)^2} = 0.67$$

Then finally compute the similarity between items 3 and 2.

$$\text{Similarity}(\text{Item 3}, \text{Item 2}) = \frac{0.33}{0.67} = \mathbf{0.5}$$

The same calculation is done for the similarity between items 3 and 4:

$$\text{Target item mean} = \frac{4 + 3}{2} = 3.5$$

$$\text{Item 4 mean} = \frac{5 + 4}{2} = 4.5$$

$$\text{Target item mean centered vector} = [(4 - 3.5), (3 - 3.5)] = [0.5, -0.5]$$

$$\text{Item 4 mean centered vector} = [(5 - 4.5), (4 - 4.5)] = [0.5, -0.5]$$

$$\text{Numerator} = (0.5 * 0.5) + (-0.5 * -0.5) = 0.5$$

$$\text{Denominator} = \sqrt{0.5^2 + (-0.5)^2} * \sqrt{0.5^2 + (-0.5)^2} = 0.5$$

$$\text{Similarity}(\text{Item 3}, \text{Item 4}) = \frac{0.5}{0.5} = \mathbf{1}$$

Next, we calculate the mean for each item, considering all the item's ratings:

$$\text{Item 2 mean} = \frac{4 + 4 + 1 + 2 + 3}{5} = \mathbf{2.8}$$

$$\text{Item 3 mean} = \frac{4 + 3 + 3}{3} = \mathbf{3.33}$$

$$\text{Item 4 mean} = \frac{5 + 3 + 4}{3} = \mathbf{4}$$

Then, we can plug in the values we found together with Madison's ratings for Items 2 and 4 (1 and 3, respectively) in the equation below:

$$\hat{r}_{ui} = \mu_i + \frac{\sum_{j \in N_u^k(i)} \text{sim}(i, j) \cdot (r_{uj} - \mu_j)}{\sum_{j \in N_u^k(i)} \text{sim}(i, j)}$$

So,

$$\text{Rating}(\text{Madison}, \text{Item 3}) = 3.33 + \frac{0.5 * (1 - 2.8) + 1 * (3 - 4)}{0.5 + 1} = \mathbf{2.07}$$

Since ratings are discrete numbers, we round this value to 2. It's important to note that in a real-world setting, it's often recommended to use neighborhood methods only when k is above a certain threshold because, when the number of neighbors is small, the predictions are usually not precise. An alternative would be to use Content-based filtering when we do not have enough data about the user-item relationship.

Online and Offline Phases

Neighborhood-based methods separate the computations into two phases: offline, where the model is fitted; and online, where inferences are made. In the offline phase, the user-user (or item-item) similarity values are precomputed, and the k most similar users or items are predetermined. These pre-computed values are leveraged to make fast predictions during the online phase.

Although good online performance is a major benefit of these methods, there is also a known disadvantage: the similarity matrix can get huge depending on the number of users/items in the system, causing the offline phase to not scale well. In addition to that, as the methods do not traditionally adapt to change, they need to update the precomputed similarities and nearest neighbors to account for new users and items, which makes the retraining process even more challenging.

The Cold-Start Problem

As previously stated, the effectiveness of Neighborhood-based CF algorithms depends on user-item interaction data. However, the user-item matrix is often sparse, with many missing values, as most users only interact with a small fraction of the items. This sparsity can lead to inaccurate recommendations due to the small neighborhood size.

This challenge is known as the cold-start problem, which is more apparent when new users or items enter the system. In this setting, the algorithm does not have enough data to form the nearest neighbors, so, the system cannot make useful recommendations.

Another important property of the user-item matrix is that the distribution of ratings among items displays a long-tail pattern. This means that only a small subset of items receives a significant number of ratings and are considered popular, while most items receive few ratings or no ratings at all. As a result, it is difficult to make precise predictions about items in the long tail using these methods, and that can be a problem because items that are less rated may provide large profit margins. This is something that is explored by Chris Anderson in his book, "The long tail". This problem can also result in a lack of diversity in recommendations because the algorithm will usually only recommend popular items.

To address these limitations to some extent, alternative algorithms can be used, such as matrix factorization and hybrid algorithms, which combine CF with Content-based Filtering methods. The next blog posts of this series will explore these topics in greater detail.

user-based and item-based CF

Collaborative Filtering (CF) is a popular technique in recommendation systems that helps predict a user's preferences by leveraging the preferences of other users or items. There are two main types of collaborative filtering: user-based and item-based.

1. **User-Based Collaborative Filtering:**

- **Idea:** This approach relies on the assumption that users who have agreed in the past tend to agree again in the future. In other words, it recommends items to a user based on the preferences of users with similar tastes.
- **Workflow:**
 1. **Similarity Calculation:** Measure the similarity between users based on their historical interactions or preferences. Common similarity metrics include cosine similarity, Pearson correlation, or Jaccard similarity.
 2. **Neighborhood Selection:** Identify a subset of users (neighborhood) who are most similar to the target user.
 3. **Prediction:** Predict the target user's preference for a particular item by aggregating the preferences of the selected neighborhood. This can be done by taking a weighted average of their ratings, for example.
- **Advantages:**
 - Intuitive approach based on the idea of finding like-minded users.
 - Easily interpretable.
- **Challenges:**
 - Cold-start problem for new users.
 - Scalability issues with a large user base.
 - Sparsity of data can lead to unreliable predictions.

2. **Item-Based Collaborative Filtering:**

- **Idea:** This approach focuses on the similarity between items rather than users. It recommends items to a user based on the similarity between the items the user has liked or interacted with in the past.
- **Workflow:**
 1. **Similarity Calculation:** Measure the similarity between items based on the users who have interacted with them. Similarity metrics are typically the same as those used in user-based CF.
 2. **Neighborhood Selection:** Identify a subset of items that are most similar to the target item.
 3. **Prediction:** Predict the target user's preference for a particular item based on their historical preferences for similar items. This is done by aggregating the ratings of the selected neighborhood.
- **Advantages:**

- Overcomes some of the scalability issues associated with user-based CF.
- Works well with a large number of users.
- **Challenges:**
 - Cold-start problem for new items.
 - May not capture user preferences as effectively as user-based CF in certain situations.

Comparison:

- **Scalability:** Item-based CF tends to scale better with a large number of users, while user-based CF can face challenges.
- **Sparsity:** Both approaches can suffer from sparsity issues, where the user-item interaction matrix is mostly empty.
- **Performance:** The performance of user-based or item-based CF depends on the characteristics of the dataset and the specific application.

components of neighbourhood methods (rating normalization, similarity weight computation, and neighbourhood selection.

Neighborhood methods in collaborative filtering involve several key components, including rating normalization, similarity weight computation, and neighborhood selection. Let's delve into each of these components in detail:

1. Rating Normalization:

Rating normalization is a crucial step in collaborative filtering systems, particularly in user-based collaborative filtering. Its primary purpose is to address variations in individual users' rating scales and tendencies, making the recommendations more accurate and robust. There are various methods for rating normalization, but one common approach is mean centering.

Mean Centering:

- **Objective:** Subtracting the mean rating of a user (or item) from each of their ratings.
- **Formula:** Normalized Rating (r') = Rating (r) - Mean Rating of the User (or Item)
- **Example:**
 - Let's consider User A and their ratings for movies: [5, 4, 3, 5, 2]. The mean rating for User A is $(5+4+3+5+2)/5 = 3.8$.
 - The normalized ratings would be: [1.2, 0.2, -0.8, 1.2, -1.8].

Purpose of Rating Normalization:

1. **Bias Correction:** Users may have different rating scales. Some users might generally rate items higher or lower than others. Rating normalization helps in mitigating these biases.
2. **Focus on Relative Preferences:** By centering ratings around the user's mean, the collaborative filtering algorithm focuses on capturing the relative preferences of the user rather than absolute values.

3. **Improved Model Performance:** Normalized ratings provide a more consistent basis for measuring similarity between users or items, leading to more accurate predictions.

Z-Score Normalization: In addition to mean centering, another normalization technique is Z-score normalization. This involves scaling the ratings by the standard deviation of a user's (or item's) ratings.

- **Formula:** Normalized Rating (r') = (Rating (r) - Mean Rating of the User) / Standard Deviation of Ratings of the User

When to Use:

- Z-score normalization is particularly useful when dealing with users who have a wide range of rating scales or exhibit extreme rating behaviors.

Example:

- If a user tends to give ratings that are consistently higher or lower than the average, Z-score normalization will scale those ratings based on how much they deviate from the mean.

2. Similarity Weight Computation:

Similarity weight computation is a key component in collaborative filtering algorithms, determining the degree of similarity between users or items based on their historical preferences. The computed similarity weights guide the recommendation system in identifying the most relevant neighbors for making predictions or recommendations. Several similarity metrics can be employed in this process, and the choice of metric depends on the characteristics of the data and the requirements of the recommendation system. Here, we'll explore common similarity metrics and their application:

Common Similarity Metrics:

1. Cosine Similarity:

- **Formula:**

$$\text{Cosine Similarity}(\mathbf{A}, \mathbf{B}) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \cdot \|\mathbf{B}\|}$$

- Measures the cosine of the angle between two vectors representing user or item preferences.
- Appropriate for scenarios where the magnitude of the vectors is essential.

2. Pearson Correlation:

- **Formula:**

$$\text{Pearson Correlation}(\mathbf{A}, \mathbf{B}) = \frac{\text{Cov}(\mathbf{A}, \mathbf{B})}{\sigma_{\mathbf{A}} \cdot \sigma_{\mathbf{B}}}$$

- Measures the linear correlation between two vectors, considering both the magnitude and direction of the ratings.
- Suitable for situations where the absolute ratings are significant.

3. Jaccard Similarity:

- **Formula:**

$$\text{Jaccard Similarity}(\mathbf{A}, \mathbf{B}) = \frac{\text{Number of Common Elements}}{\text{Total Number of Elements in Sets A and B}}$$

- Used for binary preference data (like item liked/disliked).
- Ignores non-overlapping elements in the sets.

Example:

Let's consider two users, User A and User B, and their rated items:

- User A: [5, 4, 3, 5, 2]
- User B: [4, 3, 2, 5, 1]

Using cosine similarity, the dot product of the vectors would be $(5 \times 4) + (4 \times 3) + (3 \times 2) + (5 \times 5) + (2 \times 1) = 92$. The magnitude of the vectors $\|\mathbf{A}\| = \sqrt{119}$ and $\|\mathbf{B}\| = \sqrt{55}$.

Therefore, the cosine similarity would be $\frac{92}{\sqrt{119} \times \sqrt{55}}$.

Purpose of Similarity Weight Computation:

- Neighbor Identification:** Computes a quantitative measure of similarity to identify users or items that are most similar to the target user or item.
- Weighted Aggregation:** The similarity weights serve as weights in weighted averages or other aggregation methods when predicting a user's preference for an item.
- Personalization:** Allows the recommendation system to personalize recommendations based on the preferences of similar users or items.

3. Neighborhood Selection:

Neighborhood selection is a critical step in collaborative filtering algorithms, where the goal is to identify a subset of users or items (the neighborhood) that are most similar to the target user or item. This selected subset is then used to make predictions or recommendations for the target user. The neighborhood selection process involves deciding which users or items to include in the neighborhood and, in some cases, setting a limit on the number of neighbors to consider.

Methods of Neighborhood Selection:

1. **Top-N Neighbors:**

- *Objective:* Select the N most similar users or items based on the computed similarity weights.
- *Process:* Rank all potential neighbors based on their similarity weights and select the top N for inclusion in the neighborhood.
- *Example:* If N is set to 10, the top 10 most similar users to the target user form the neighborhood for making recommendations.

- *Purpose:* Focuses on the most similar entities, ensuring a balance between accuracy and computational efficiency.

2. **Threshold-Based Selection:**

- *Objective:* Include only those users or items with a similarity score above a certain threshold.
- *Process:* Set a similarity threshold, and include users or items in the neighborhood only if their similarity score surpasses this threshold.
- *Example:* If the similarity threshold is set to 0.8, only users or items with a similarity score of 0.8 or higher are included in the neighborhood.
- *Purpose:* Provides more flexibility in the size of the neighborhood and can be used to filter out less relevant or less similar entities.

Considerations in Neighborhood Selection:

1. **Computational Complexity:**

- Selecting too many neighbors can lead to increased computational complexity, especially in large datasets.
- Balancing the number of neighbors to include is crucial for achieving a trade-off between accuracy and efficiency.

2. **Sparsity of Data:**

- In sparse datasets, where users have only interacted with a small fraction of items, it may be challenging to find a sufficient number of neighbors.
- Threshold-based methods can be useful in such scenarios.

3. **Impact on Cold Start:**

- Neighborhood selection methods should account for the "cold start" problem, where new users or items have limited interaction history.
- In such cases, alternative methods like content-based recommendations or hybrid models may be employed.

Example:

Let's consider a user-based collaborative filtering scenario. If User A is the target user, the neighborhood selection process may involve computing similarity weights with all other users and selecting the top 10 most similar users as neighbors for User A.

Purpose of Neighborhood Selection:

1. **Relevance:** Ensures that the selected neighbors are the most relevant and similar entities to the target user or item.
2. **Computational Efficiency:** Manages computational complexity by limiting the number of neighbors, balancing accuracy with efficiency.
3. **Personalization:** Allows the recommendation system to tailor recommendations based on the preferences of a select group of similar users or items.

